

Data Science Lab in R - Lab One

2025-06-18

Short Answer Questions

For the following questions, please response in 3-4 sentences.

Question 1

Give an example of how search engines can cause confirmation bias. Discuss how to possible combat this.

When websites project biased opinions and more people search/click the link, search engines could place that website higher up on the list, making more users click on the following biased-website. This could spread around biased ideas making it seem as if the ideas are facts.

Question 2

Read this article about Amazon's customer analysis. Discuss the pros and cons of using race in the analysis. Do you agree?

The pro of using race in the analysis is it brings up a minority or an outlier that was not previously covered by Amazon's data. This kind of mirrors the idea of how when looking at an average salary between two nearby neighborhoods, factors such as Bill Gates living in one of the neighborhood the entire average could be shifted. A con of using race is that it oversimplifies the situation without including external factors such as income, population, and spending patterns. Instead, of Amazon being biased toward urban communities it might have actually just been a more economic issue on where they could gain money more.

Question 3

This might be of particular interest to you: Why an end-of-the alphabet last name could skew your grades. Discuss the pros and cons from your own student perceptive and a hypothetical teacher perspective.

A pro of this research study is that it could have identified a potential bias in name ordering with the grades and how it can influence academic outcomes. A con, however, is the idea of correlation is not causation. Although grades at the lower-leveled last names are lower that does not directly mean that teachers are simply marking off points at the end in a rush.

Question 4

How can your *overall* error rate be low, but your errors are unequally distributed across certain groups?

An overall error rate can be low while have unequal distribution of error because of the way means work. If at one group the error is very high but in another very very little, the error would lean to be lesser making the overall error rate low.

Question 5

Of the ethical issues in data science discussed so far, what concerns you the most, and why?

The biggest ethical issue in data science are the misconceptions that arise from correlation with people thinking it is causation. Many misconceptions as such are spread just from facebook articles themselves with many believing whatever they see on social media. This could hurt many companies such as the prior example of speculations with Amazon and its bias towards black communities.

Question 6

What do you think makes a graphic easy to interpret and visually appealing?

Colors can make graphics easy to interpret so that it is easy to quickly identify different categories of data. A major attribute that could make a graphic easy to interpret is whichever way it could show the trend of the data so we could see the correlation between two components– manipulating it for intel.

Coding Questions

For each question below, if you create a variable or an R object, please print out the variable after creating.

Click “Knit” above regularly to generate a pdf of this document. If you only knit the document at the end, it’s much more difficult to troubleshoot.

Section 1: Navigating RStudio

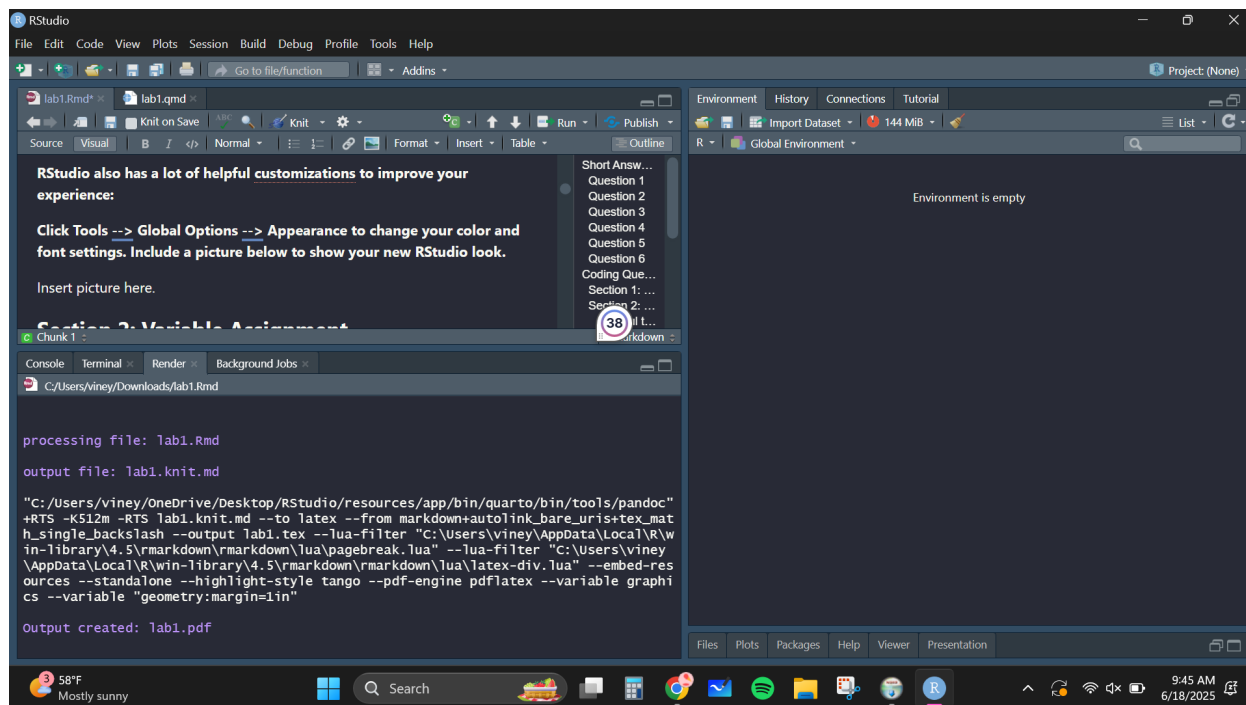
Load the tidyverse library. You should always do this before starting to code to make sure you have several helpful packages at your disposal, like ggplot2 and dplyr.

```
library(tidyverse)
```

RStudio also has a lot of helpful customizations to improve your experience:

Click Tools -> Global Options -> Appearance to change your color and font settings. Include a picture below to show your new RStudio look.

Insert picture here.



Section 2: Variable Assignment

Try assigning the value “5” to the variable ‘number’ using the assignment arrow `<-` and then run ‘number’.

```
number <- 5
```

Assign a variable called `string` the value “hello, world” using the ‘`=`’ operator to do variable assignment.

```
string = "hello, world"
```

There are subtle differences between ‘`<-`’ and ‘`=`’, which won’t matter in most cases. However, using ‘`<-`’ is considered good code style. You want your code to adhere to good stylistic practices, since that makes it easier to read and use by other users.

Perform basic arithmetic using the variable `number` in R below, including:

- addition
- subtraction
- multiplication
- division

```
number + 2
```

```
## [1] 7
```

```
number * number
```

```
## [1] 25
```

```
number * 3
```

```
## [1] 15
```

```
number/4
```

```
## [1] 1.25
```

Use a hashtag # to comment your code (e.g., write notes to your future self and your collaborators) to help keep your script organized. Add a comment below.

```
#this is so aura
```

Helpful tricks: To run a line of code, place your cursor on a line of runnable code and click the “Run” button or click Ctrl + Enter (PC) or command + Enter (Mac).

Press Ctrl + l on both Mac and PC to clear your console. This does not erase any data. It simply moves the console prompt (the greater-than symbol: >) up to the top to improve readability.

Section 3: Data Classes and Coercion

There are five main classes of data we will work with in this workshop:

1. **numeric:** decimals (the default for ALL numbers in R).
2. **integer:** whole numbers (positive and negative, including zero).
3. **character:** text strings (always wrapped in quotations).
4. **logical:** TRUE or FALSE (1 or 0).
5. **factor:** nominal or ordinal categorical type.

A factor variable is a set of categorical or ordinal values. To create a factor variable, we first need a vector. We’ll return to factors in the next section after we’ve learned about vectors.

Like `as.integer`, other “as dot” functions exist as well, such as `as.numeric`, `as.character`, `as.logical`, and `as.factor`.

1. Define three variables: one numeric, one character, and one logical

```
numeric_var <- 10  
char_var <- "hello"  
log_var <- FALSE
```

- 2a. Can you convert numeric to integer type?

```
as.integer(number)
```

```
## [1] 5
```

2b. Convert numeric to logical?

```
as.logical(numeric_var)
```

```
## [1] TRUE
```

2c. Convert numeric to character?

```
as.character(number)
```

```
## [1] "5"
```

2d. Convert logical to character?

```
as.character(log_var)
```

```
## [1] "FALSE"
```

2e. Convert character to numeric?

```
as.numeric("A")
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA
```

```
#not possible
```

2f. Rank the hierarchy of the following data types: character, integer, numeric, boolean/logical.

character < numeric < integer < boolean/logical

Section 4: Data Structures

There are four main functions you can use to form more complex data structures:

1. `c()`: ordered groupings of the SAME type of data (called “vectors”).
2. `data.frame()`: and ordered group of equal-length vectors; also like a list. We’ll work with these in Section 5.
3. `list()`: store heterogeneous data.
4. `matrix()`: like a vector, but can be ordered into rows and columns.

Helpful tip: Type a question mark before a function name to view the help pages. Read the “Description” section for a definition. The “Usage” section shows what arguments can be passed into the function and the “Arguments” section provides definitions of the individual parameters/arguments. Also see the “Details”, “References”, and “Examples” sections for more information.

Section 4.1: Vectors

A vector is an ordered group of the *same* type of data. We can create vectors by concatenating data together with the `c()` function. We can also store missing values in a vector using the `NA` keyword:

Create a vector called `vec` with the following values: 2,5,8,NA,14.

```
vec = c(2,5,8,NA,14)
vec
```

```
## [1] 2 5 8 NA 14
```

Taking the mean while dropping NAs and while not dropping NAs using `mean()`. Note the difference.

```
mean(vec)
```

```
## [1] NA
```

```
mean(vec, na.rm=TRUE)
```

```
## [1] 7.25
```

#the difference is that when the mean is used

Create a vector called `vec2` using `seq()`. Take a trimmed mean and get the length and class of the vector using `length()` and `class()`.

```
vec2= seq(to=0, from=100, by=-10)
vec2
```

```
## [1] 100 90 80 70 60 50 40 30 20 10 0
```

Create a character vector of length 3 called `char_vec` and a logical vector of length 4 called `log_vec`.

```
char_vec <- c("ishanvi", "aura", "ridhima")
char_vec
```

```
## [1] "ishanvi" "aura" "ridhima"
```

Challenge 1: Concatenating vectors

1. Create a vector called `vec1` starting from 10, and ending with 30, with each entry separated by 2.

```
vec1 = seq(to=30, from=10, by=2)
vec1
```

```
## [1] 10 12 14 16 18 20 22 24 26 28 30
```

2. Index the 3rd through 7th elements of this vector. Hint: use `[]` and `:`.

```
vec1[3:7]
```

```
## [1] 14 16 18 20 22
```

3. Index the 3rd through 7th elements and the 10th element. Hint: use `c()`.

```
vec1[c(3:7, 10)]
```

```
## [1] 14 16 18 20 22 28
```

Section 4.2: Lists

A list can be used to store heterogeneous data.

Create a list called `three_types` using `list()` with three data types: logical, character, and integer.

```
three_types = list(FALSE, "aura", 5)
```

Section 4.3: Matrices

Create a matrix called `one_through_six` that has the numbers 1 through 6 (hint: using `:`) with 2 rows and 3 cols.

```
one_through_six <- matrix(data = 1:6, nrow = 2, ncol = 3)
one_through_six
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

Section 5: Dataframes and Functions

Create a dataframe called `classmates` with yourself and 5 of your peers (6 rows in total) with the following variables: `name`, `age`, `hometown`, `favorite_color` and `coding_exp` (TRUE/FALSE). Hint: start by creating vectors for each column.

```
name <- c("ishanvi", "ridhima", "dannah", "tara", "sowjhaniya", "jady")
age <- c(16, 16, 16, 15, 17, 15)
hometown <- c("niles", "fremont", "fremont", "san francisco", "fremont", "santa clara")
color <- c("pink", "purple", "yellow", "purple", "red", "black")
coding_exp <- c(T, T, T, T, T, T)
classmates <- data.frame(name, age, hometown, color, coding_exp)
```

Functions perform actions on inputs. They are followed by trailing round parentheses.

Arguments are the inputs - values, expressions, text, entire datasets, etc. You tell a function what arguments it needs inside the parentheses. Sometimes, these arguments are “named”. This is helpful when you need to enter multiple arguments: the names tell R which arguments correspond to what variables you’re passing into the function.

Use `mutate()` to create a new column called `age_in_months` and save it back into `classmates` using the assignment arrow `<-`.

```
classmates |>
  mutate(age_in_months = age*12)
```

```
##      name age      hometown  color coding_exp age_in_months
## 1  ishanvi  16        niles   pink      TRUE          192
## 2  ridhima  16      fremont purple      TRUE          192
## 3   dannah  16      fremont yellow      TRUE          192
## 4    tara  15 san francisco purple      TRUE          180
## 5 sowjhaniya 17      fremont   red      TRUE          204
## 6     jady  15   santa clara black      TRUE          180
```

Use `filter()` to show all people who are older than 16 in `classmates`.

```
classmates |>
  filter(age>=16)
```

```
##      name age hometown  color coding_exp
## 1  ishanvi  16   niles   pink      TRUE
## 2  ridhima  16 fremont purple      TRUE
## 3   dannah  16 fremont yellow      TRUE
## 4 sowjhaniya 17 fremont   red      TRUE
```

Use the `$` operator, `mean()` and `median()` to get the mean and median age of all people in `classmates`.

```
mean(classmates$age)
```

```
## [1] 15.83333
```

```
median(classmates$age)
```

```
## [1] 16
```

Use `summarize()` to get the mean and median age of all people in `classmates`. Check that you get the same values as above.

```
classmates |>
  summarize(mean_age = mean(age),
            median_age = median(age),
            sd = sd(age),
            IQR = IQR(age))
```

```
##  mean_age median_age      sd  IQR
## 1 15.83333         16 0.7527727 0.75
```

Use `group_by()` and `summarize()` to get the mean and standard deviation of people who have coding experience and people who do not.


```
classmates |>
  group_by(coding_exp) |>
  summarize(mean_age = mean(age),
            median_age = median(age),
            sd = sd(age),
            IQR = IQR(age))
```

```
## # A tibble: 1 x 5
##   coding_exp mean_age median_age    sd    IQR
##   <lgl>      <dbl>      <dbl> <dbl> <dbl>
## 1 TRUE          15.8          16 0.753  0.75
```

Pick two towns in the hometown column. Use the `filter()` and `%in%` to show only students from those two towns.

```
classmates |>
  filter(hometown%in%
         c("niles", "fremont"))
```

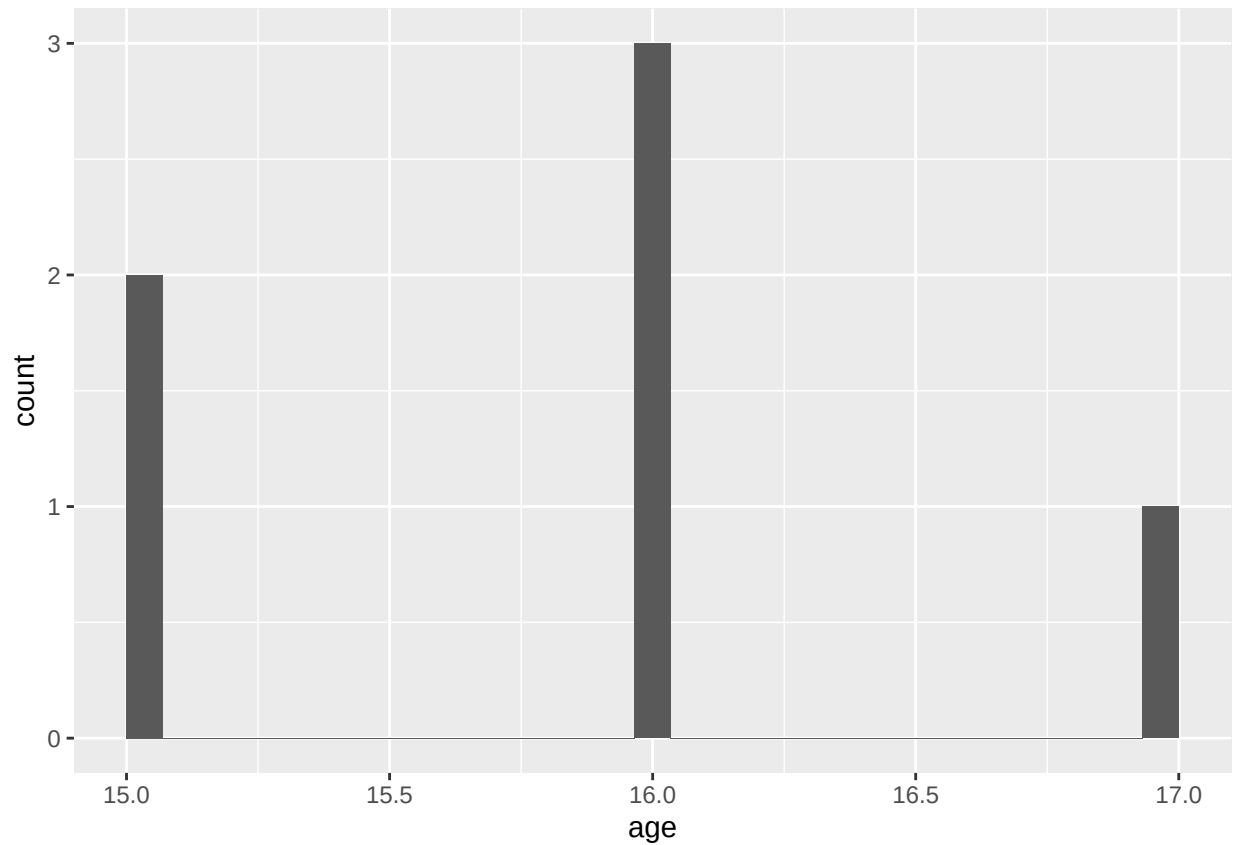
```
##      name age hometown  color coding_exp
## 1  ishanvi  16    niles   pink        TRUE
## 2  ridhima  16  fremont purple        TRUE
## 3   dannah  16  fremont yellow        TRUE
## 4 sowjhaniya 17  fremont   red        TRUE
```

Section 6: Plotting

Use `ggplot()` for the following section. You may reuse variables across different plots.
Create a histogram using one of the variables from `classmates`.

```
classmates |>
  ggplot(aes(x=age)) + geom_histogram()
```

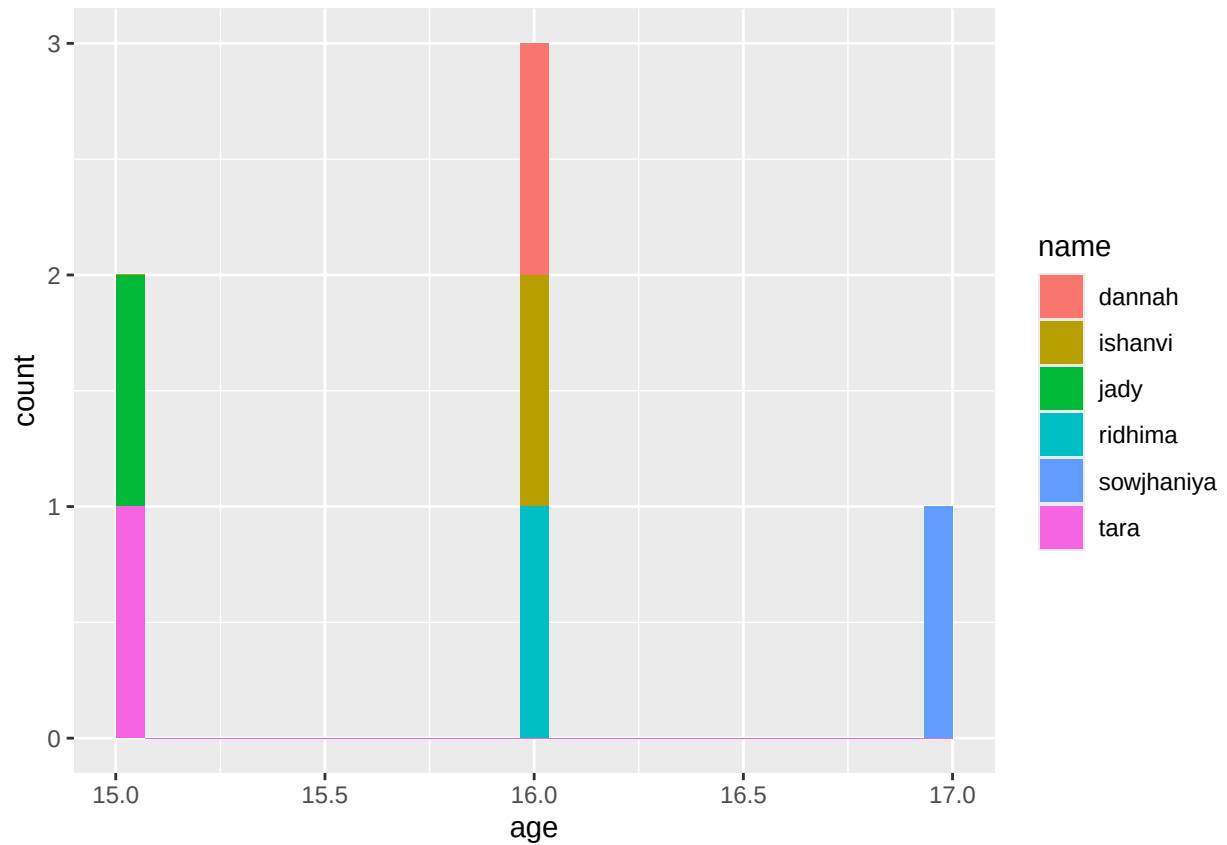
```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



Add a variable to the histogram above using the fill aesthetic.

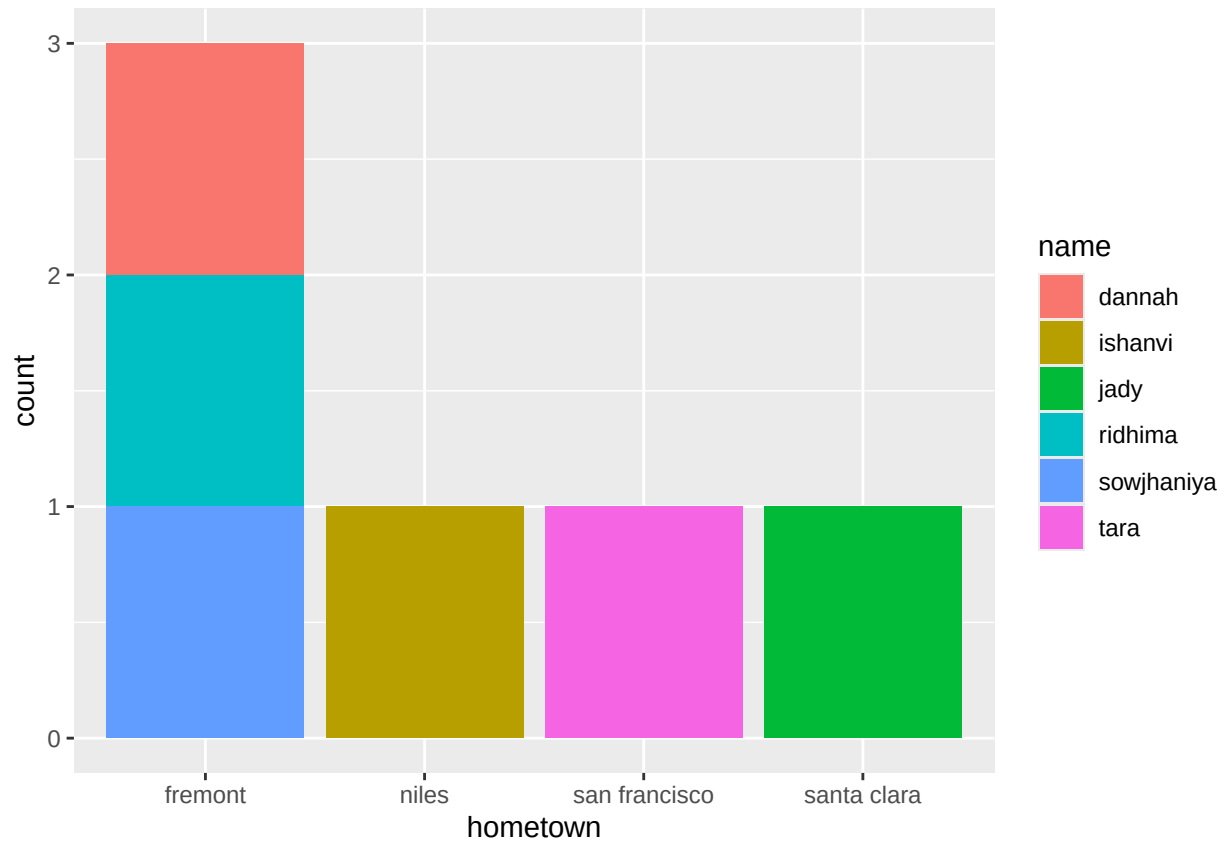
```
classmates |>  
  ggplot(aes(x=age,fill=name)) + geom_histogram()
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



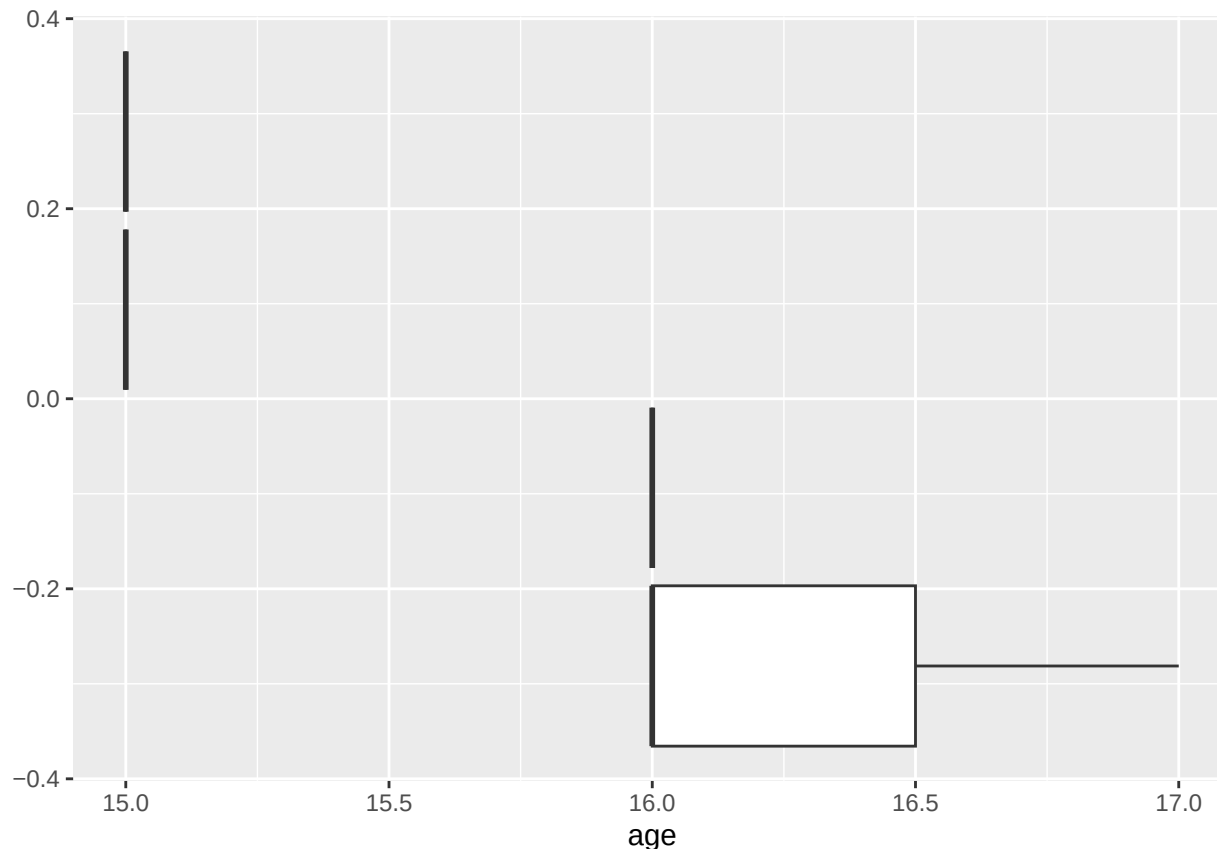
Create a bar chart using one of the variables from `classmates`.

```
classmates |>  
  ggplot(aes(x=hometown, height=name, fill=name))+  
  geom_bar()
```



Create a boxplot using one of the variables from classmates.

```
classmates |>  
  ggplot(aes(x=age, height=hometown))+  
  geom_boxplot()
```



In a few sentences, what have you learned about the demographics of your peers?

Written answer here.

I have learned that many of us are clustered in the same area with the same age. As well as, seen from the boxplot the median is 16 and the mean is 15.8333333 which means most of us are 16. By the boxplot we can see outliers at 15 and 17.

Section 7: Multiple Dataframes

Let's make a toy dataframe:

```
df1 <- data.frame(
  Name = c("Joe", "Susan", "Jack", "Kelly"),
  City = c("Berkeley", "Berkeley", "Oakland", "Oakland"),
  Math = c(42, 48, 50, 46),
  Reading = c(8, 10, 10, 10))
df1
```

```
##   Name      City Math Reading
## 1   Joe Berkeley  42      8
## 2 Susan Berkeley  48     10
## 3  Jack  Oakland  50     10
## 4 Kelly  Oakland  46     10
```

And a second toy dataframe:

```
df2 <- data.frame(
  Name = c("Joe", "Susan", "Jack", "Kelly"),
  Science = c(99, 100, 99, 100),
  Music = c(19, 18, 20, 20),
  Art = c(20, 20, 19, 18))
df2
```

```
##   Name Science Music Art
## 1  Joe      99    19  20
## 2 Susan    100    18  20
## 3  Jack     99    20  19
## 4 Kelly    100    20  18
```

Merge the dataframes using `merge()` and the `Name` column. Call this new dataframe `df_merged`.

```
df_merged <- merge(df1, df2, by = "Name")
df_merged
```

```
##   Name      City Math Reading Science Music Art
## 1  Jack  Oakland   50     10     99    20  19
## 2   Joe Berkeley   42      8     99    19  20
## 3 Kelly  Oakland   46     10    100    20  18
## 4 Susan Berkeley   48     10    100    18  20
```

In `dplyr`, there are several “join” functions used to perform merges. These include `inner_join()`, `left_join()`, `right_join()`, and `full_join()`. It’s very easy to mix these up and it usually takes some trial and error to find the right one. Check out this site for some helpful animations of the different kinds of joins.

The variations consist of how to treat cases when there might not be a one-to-one mapping between two rows of a data frame.

Try using `full_join()` to join `df1` and `df2`. Call this new dataframe `df_joined`. (It should be basically the same as above!)

```
df_joined <- full_join(df1, df2)
```

```
## Joining with 'by = join_by(Name)'
```

```
df_joined
```

```
##   Name      City Math Reading Science Music Art
## 1   Joe Berkeley   42      8     99    19  20
## 2 Susan Berkeley   48     10    100    18  20
## 3  Jack  Oakland   50     10     99    20  19
## 4 Kelly  Oakland   46     10    100    20  18
```

Challenge 2

Consider the following data frame:

```
challenge_df <- data.frame(
  Name = c("Joe", "Susan", "Juan", "Preeti"),
  City = c("Berkeley", "Berkeley", "San Mateo", "San Jose"),
  Physics = c(20, 21, 22, 23),
  Chemistry = c(23, 22, 21, 20))
```

1. Use the `merge()` function to merge `df_merge` and `challenge_df` by Name. What do you notice about the columns? How many rows are there?

Written answer here.

```
df_joined <- merge(df1, df2)
df_joined
```

```
##      Name      City Math Reading Science Music Art
## 1  Jack  Oakland   50      10      99    20  19
## 2   Joe Berkeley   42       8      99    19  20
## 3 Kelly  Oakland   46      10     100    20  18
## 4 Susan Berkeley   48      10     100    18  20
```

2. Use the `inner_join()` function from `dplyr` to merge `df_merge` and `challenge_df` by Name. What happens if you merge by Name and City?

Written answer here.

```
df_joined <- inner_join(df_merged, challenge_df, by=c("Name", "City"))
df_joined
```

```
##      Name      City Math Reading Science Music Art Physics Chemistry
## 1   Joe Berkeley   42       8      99    19  20      20      23
## 2 Susan Berkeley   48      10     100    18  20      21      22
```

3. Use the `left_join()` function to merge `df_merge` and `challenge_df` by Name and City. Use the `right_join()` function to merge `df_merge` and `challenge_df` by Name and City.

Written answer here.

```
df_ljoined <- left_join(df_merged, challenge_df, by=c("Name", "City"))
df_ljoined
```

```
##      Name      City Math Reading Science Music Art Physics Chemistry
## 1  Jack  Oakland   50      10      99    20  19      NA      NA
## 2   Joe Berkeley   42       8      99    19  20      20      23
## 3 Kelly  Oakland   46      10     100    20  18      NA      NA
## 4 Susan Berkeley   48      10     100    18  20      21      22
```

```
df_rjoined <- right_join(df_merged, challenge_df, by=c("Name", "City"))
df_rjoined
```

```
##      Name      City Math Reading Science Music Art Physics Chemistry
## 1  Jack  Oakland   50      10      99    20  19      NA      NA
## 2   Joe Berkeley   42       8      99    19  20      20      23
## 3 Kelly  Oakland   46      10     100    20  18      NA      NA
## 4 Susan Berkeley   48      10     100    18  20      21      22
```

4. Use the `full_join()` function to merge `df_merge` and `challenge_df` by Name and City.

```
df_joined <- full_join(df_merged, challenge_df, by=c("Name", "City"))
df_joined
```

##	Name	City	Math	Reading	Science	Music	Art	Physics	Chemistry
## 1	Jack	Oakland	50	10	99	20	19	NA	NA
## 2	Joe	Berkeley	42	8	99	19	20	20	23
## 3	Kelly	Oakland	46	10	100	20	18	NA	NA
## 4	Susan	Berkeley	48	10	100	18	20	21	22
## 5	Juan	San Mateo	NA	NA	NA	NA	NA	22	21
## 6	Preeti	San Jose	NA	NA	NA	NA	NA	23	20

5. What do you notice about the differences among 2, 3, and 4?

The differences between 2, 3, and 4 emerge from how the matrices are being merged. In scenario 2, the matrices are joined by `inner_join`, which means the common columns will be combined, while the rest will not be included. In scenario 3, the matrices are joined by `left_join` and `right_join`, `left_join` has more weight on the left column meaning when it merges all the components of the left column gets merged over even if it does not match with the right column (the components of the right column that don't get transferred over will have NA). In scenario 4, the matrices are joined by `full_join` where all the columns are joined together to make a composite matrix.

Section 8: Turning the lab in

Click the 'Knit' button above. If all of your code runs, this should compile into a pdf. Download this pdf and turn it in on Canvas for the Lab 1 Assignment.